Event Management System using MVC Architecture (Java Swing GUI)

Institute: institute of engineering and management

Department: Information Technology (IT)

CERTIFICATE

This is to certify that **Debdatta Roychowdhury** (Roll No. **44**), a student of **Section – A** from the **Department of Information Technology**, **Institute of Engineering and Management, Kolkata**, has successfully completed the project titled **"Event Management System using MVC Architecture (Java Swing GUI)"** during the academic year **2025–26** under the supervision and guidance of the department.

The project work presented in this report is the **original work** of the student and has not been submitted to any other institution or university for the award of any degree, diploma, or certificate.

(Signature of Guide) **(Signature of Head of Department)**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **the Institute of Engineering and Management, Kolkata**, for giving me the opportunity to work on this project titled **"Event Management System using MVC Architecture (Java Swing GUI)"**.

I am thankful to my faculty members and project guide for their valuable support, suggestions, and encouragement throughout the development of this project. Their guidance helped me understand the concepts of **Java programming, Swing GUI, MVC architecture, and object-oriented programming** in a practical and structured way.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives of the Project
4. Scope of the Project
5. Technologies Used
6. System Requirements
7. Proposed System / Architecture
8. OOP Concepts Used
9. Core Features
10. File Structure
11. Working Procedure
12. Screenshots / Demo Output
13. Advantages and Limitations
14. Future Enhancements
15. Conclusion
16. References

ABSTRACT

Managing event details manually can often become difficult, confusing, and time-consuming, especially when multiple events need to be handled at the same time. To solve this problem, this project presents a simple desktop-based **Event Management System** developed using **Java Swing GUI** and following the **MVC (Model-View-Controller) Architecture**.

The system allows users to perform the basic CRUD operations, which include **adding, viewing, editing, and deleting event records** through a user-friendly graphical interface. The project is designed in a simple and beginner-friendly way so that students can easily understand the structure and working process.

The **Model** manages event data, the **View** handles the graphical user interface, and the **Controller** manages communication between them. This separation improves code readability, maintainability, and scalability.

This project is highly useful for learning **Java GUI development, object-oriented programming concepts, event handling, and basic software architecture** in a practical manner.

1. INTRODUCTION

An Event Management System is a software application used to store and manage event-related information in an organized manner. In many cases, event details are maintained manually using notebooks or spreadsheets, which can lead to errors, duplication, and difficulty in updating records.

This project provides a simple desktop-based solution where a user can manage event records using a graphical interface. It is built using **Java Swing** for the GUI and follows the **MVC architecture**, which separates the application into data, interface, and control logic. This makes the project more structured and easier to understand.

2. PROBLEM STATEMENT

Managing event details manually can be difficult and time-consuming, especially when multiple events need to be tracked. This project solves that problem by creating a simple desktop-based Event Management System using Java and Swing. The system allows users to add, view, update, and delete event records through a graphical user interface. It follows the MVC (Model-View-Controller) architecture to separate data, user interface, and business logic, making the project easy to understand and maintain. This project is suitable for beginners to learn Java GUI development, object-oriented programming concepts, and basic CRUD operations in a structured way.

3. OBJECTIVES OF THE PROJECT

The main objectives of this project are:

- To develop a simple desktop-based Event Management System.
- To perform CRUD operations (Create, Read, Update, Delete).
- To provide a user-friendly graphical interface using Java Swing.
- To implement MVC architecture in a practical project.
- To improve understanding of OOP concepts in Java.
- To create a beginner-friendly academic project.

4. SCOPE OF THE PROJECT

The scope of this project includes:

- Managing event details in a simple desktop application.
- Adding new event records.
- Updating existing event details.
- Deleting unnecessary event records.

- View all event data in a table format.
- Demonstrating MVC architecture in practical implementation.

This project is mainly designed for educational and learning purposes.

5. TECHNOLOGIES USED

- **Programming Language:** Java
- **GUI Framework:** Java Swing
- **Architecture Pattern:** MVC (Model-View-Controller)
- **IDE Used:** VS Code / IntelliJ IDEA / Eclipse
- **JDK Version:** JDK 17 or above
- **Version Control:** Git & GitHub

6. SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB or above
- Storage: Minimum 500 MB free space

Software Requirements

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK): Version 17 or above
- IDE: VS Code / IntelliJ IDEA / Eclipse
- Terminal / Command Prompt
- Git (optional, for GitHub upload)

7. PROPOSED SYSTEM / ARCHITECTURE DESCRIPTION

The project follows the **MVC (Model-View-Controller)** architecture:

- **Model:**
  Stores the event data such as Event ID, Event Name, Date, Venue, and Organizer.

- **View:**
  Provides the graphical user interface using Java Swing components like labels, text fields, buttons, and tables.
- **Controller:**
  Connects the Model and View. It handles user actions such as Add, Edit, Delete, and Clear.

This architecture keeps the code clean and organized.

8. OOP CONCEPTS USED

- **Abstraction:**
  The event data is represented through a separate model class, hiding unnecessary implementation details.
- **Inheritance:**
  Swing classes like JFrame, JPanel, and JTable use inheritance internally.
- **Polymorphism:**
  Method overriding can be seen in action listeners and event-handling methods.
- **Exception Handling:**
  Try-catch blocks are used to prevent crashes due to invalid input or runtime issues.
- **Collections / Threads:**
  ArrayList is used to store multiple event records dynamically.

9. CORE FEATURES

- Add new event details
- View all event records in table format
- Edit/update selected event details
- Delete selected event records
- Clear input fields
- Beginner-friendly GUI
- MVC structured project

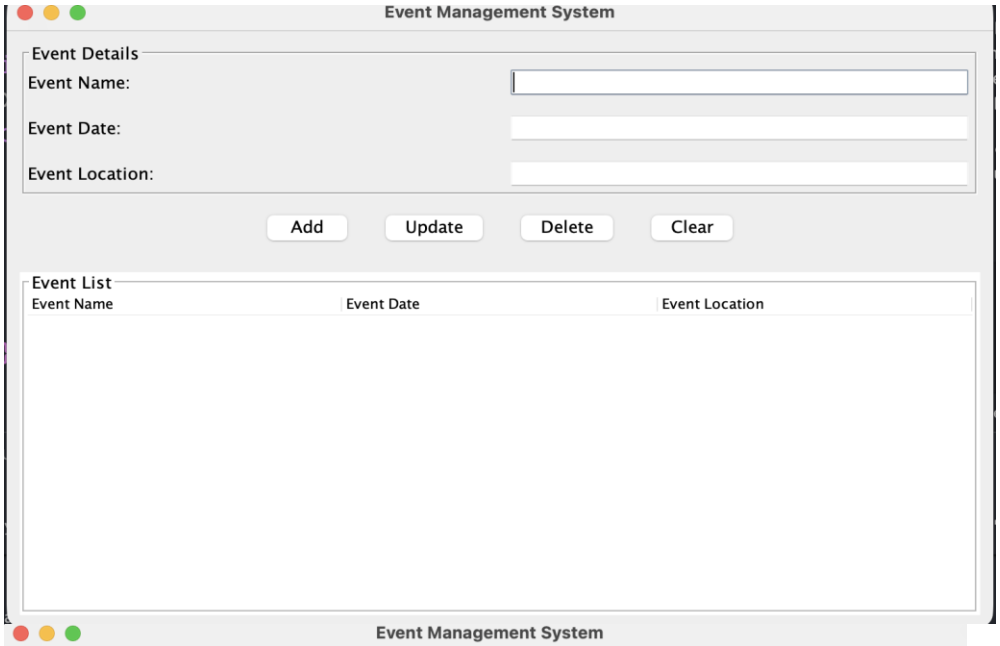
10. FILE STRUCTURE

```
EventManagementSystem/
│
├── src/
│   ├── model/
│   │   └── Event.java
│   ├── view/
│   │   └── EventView.java
│   ├── controller/
│   │   └── EventController.java
│   └── Main.java
│
└── out/        (this will be created after compile)
```

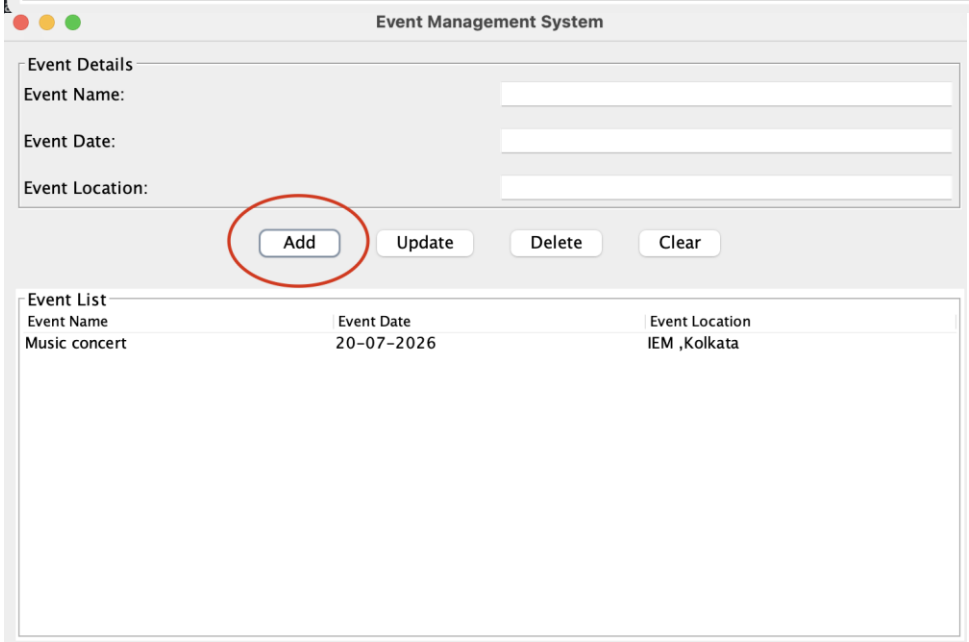
11. WORKING PROCEDURE

1. The user opens the application.
2. The GUI window appears with input fields.
3. The user enters event details.
4. Click **Add** to store the event in the table.
5. Clicking a table row loads data into the form.
6. The user can click **Update** to modify data.
7. The user can click **Delete** to remove the selected event.
8. The **Clear** button resets the form.

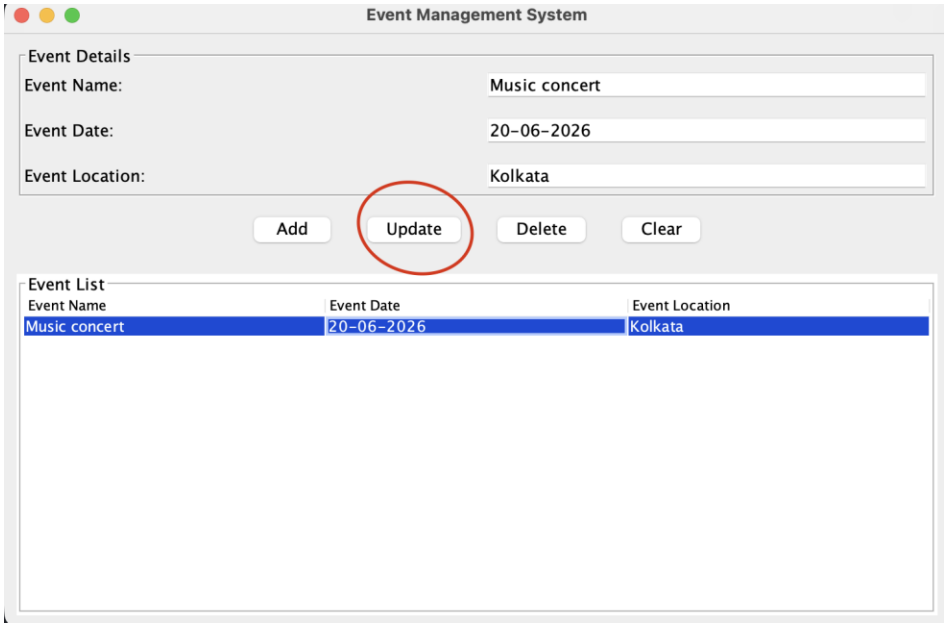
12. SCREENSHOTS / DEMO OUTPUT



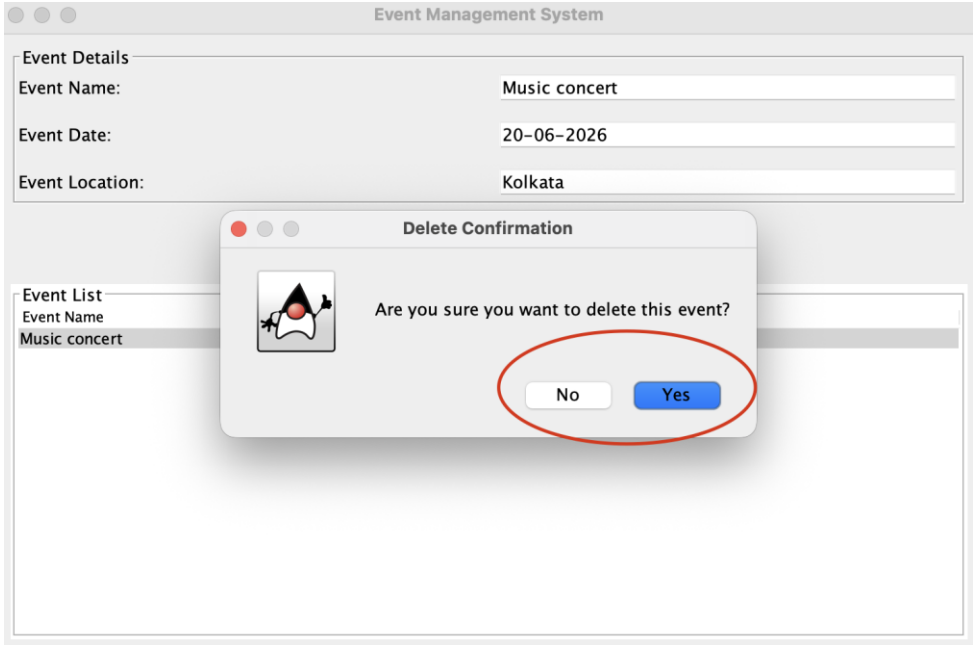
12.1 Main GUI Window



12.2 – Adding Event Data



12.3Editing Event Data



12.4 Deleting Event Data

13. ADVANTAGES

- Easy to understand for beginners
- Clean and structured code using MVC
- User-friendly GUI
- Supports full CRUD operations
- Helps in learning Java Swing practically
- Useful as a college mini project

14. LIMITATIONS

- Data is not permanently stored (if not connected to database)
- Only basic features are implemented
- No login/authentication system
- Suitable mainly for desktop use
- Limited scalability in current version

15. FUTURE ENHANCEMENTS

This project can be improved in the future by adding:

- Database connectivity (MySQL / SQLite)
- User login and authentication
- Search and filter functionality
- Export data to PDF / Excel
- Better UI design
- Date picker for event date
- Online / web-based version of the system

16. CONCLUSION

The **Event Management System using MVC Architecture (Java Swing GUI)** is a simple and effective desktop-based application developed for managing event records in an organized way. It

successfully performs the core operations of adding, viewing, updating, and deleting event data through a graphical user interface.

This project helped in understanding important programming concepts such as **Java Swing GUI, MVC architecture, object-oriented programming, event handling, and CRUD operations**. The project is especially useful for beginners because it provides a practical and structured approach to building software applications.

Overall, this project serves as a strong academic mini-project and can be further enhanced with database support and advanced features in the future.

17. REFERENCES

- Java Official Documentation
- Oracle Java Swing Documentation
- MVC Architecture Concepts
- Classroom Notes and Faculty Guidance
- GitHub (for version control and project hosting)